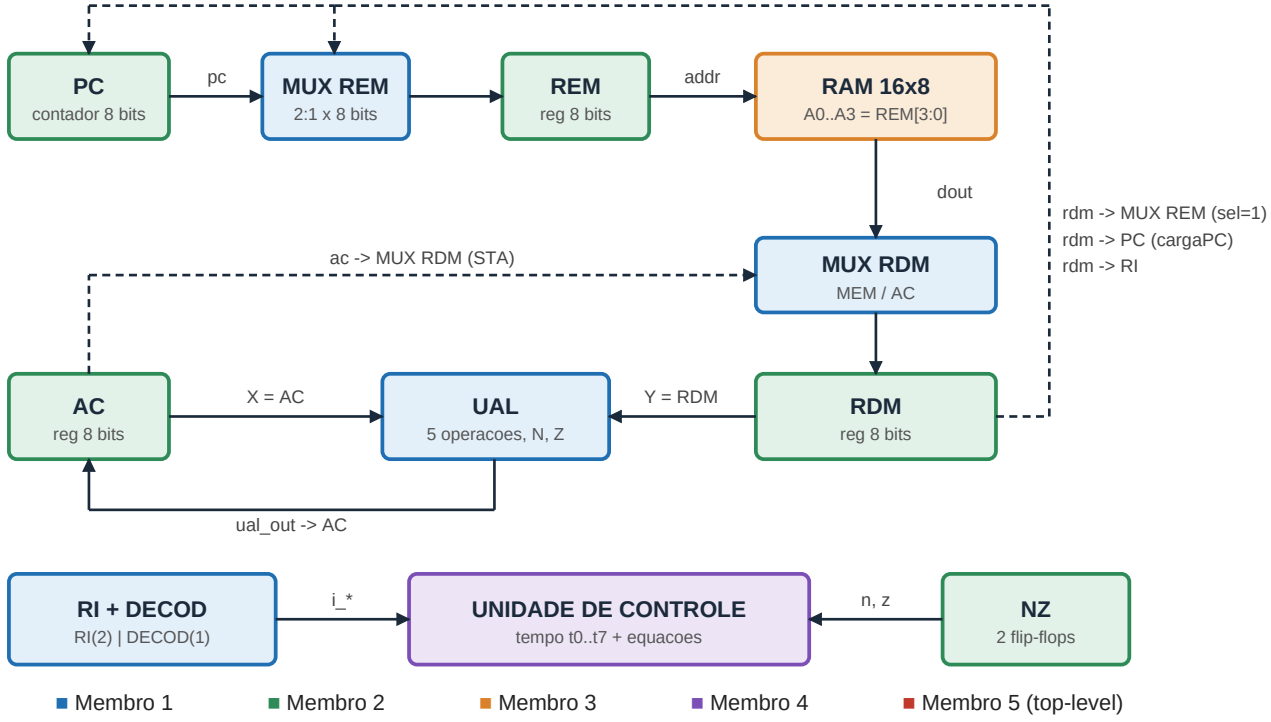
Projeto do Processador Neander

Guia de execução para o grupo de 5 pessoas

Nível de portas lógicas / Verilog estrutural · Sistemas Digitais 2026 · UNIOESTE Cascavel · Prof. Leonardo Medeiros

1. Visão geral

Objetivo: construir o processador Neander (8 bits, complemento de dois) usando apenas portas básicas (NAND, NOR, NOT), latches e flip-flops, e simulá-lo executando um programa de 10 instruções gravado na RAM. Este guia diz o que cada pessoa entrega, em qual ordem, com quais sinais e como testar. **Leia as seções 1 a 3 juntos na primeira reunião**; depois cada um segue a sua página.



Linhas tracejadas: caminhos que voltam para o início do datapath (AC → MUX RDM, RDM → MUX REM, RDM → PC, RDM → RI). A UAL tem X = AC e Y = RDM permanentemente.

Membro	Responsável por	Entrega principal	Depende de
1	Biblioteca de portas, MUX 2:1 de 8 bits, UAL, decodificador de opcode	**lib_portas.v** (dia 1), **mux2x8, ual, decod_opcode**	ninguém (começa primeiro)
2	Latch/FF, registradores AC, REM, RDM, RI, NZ e contador PC	**reg8, rdm, regnz, pc8**	M1 (portas e mux)
3	RAM 16 × 8 com decodificadores, leitura/escrita e RESET com programa	**ram16x8**	M2 (FF), M5 (tabela do programa)
4	Unidade de Controle: gerador de tempos t0–t7 e equações dos sinais	**gerador_tempo, unidade_controle**	M1 (decod.), M2 (FF)
5	Programa, modelo de referência, top-level, testbench, simulação e relatório	**neander_top, tb_neander, mem_init, relatório**	todos (integra)

2. Decisões da primeira reunião

Estes pontos precisam estar resolvidos antes de qualquer pessoa escrever circuito. Os dois primeiros dependem do professor: mandem a pergunta já no primeiro dia.

#	Decisão	Recomendação do guia	Quem fecha
1	**"10 instruções" são 10 instruções ou 10 bytes?** LDA, STA, ADD, OR, AND, JMP, JN e JZ ocupam 2 bytes. O enunciado também diz "0000 a 1010" (11 posições) e "6 endereços restantes" (dados).	Perguntar ao professor. Conta útil: com *x* instruções de 2 bytes, o código ocupa **10 + x** bytes e sobram **6 − x** para dados (se forem 10 instruções). Se forem 10 bytes de código, sobram 6 para dados.	M5 pergunta; grupo decide
2	**Verilog estrutural ou simulador de portas?** O enunciado cita os dois.	Perguntar. Se for Verilog, nos módulos do processador usar só **nand, nor, not** e fios (nada de +, *assign a = b & c*, *always* com lógica). A testbench pode ser comportamental.	M5 pergunta
3	Primitivas *and/or/xor* são permitidas?	Na dúvida, M1 monta tudo a partir de nand/nor/not na **lib_portas.v**, e todos usam só a biblioteca.	M1
4	Clock e reset	Borda de **subida**. Um tempo t_k por ciclo de clock. **RESET síncrono, ativo em 1**, mantido por pelo menos 2 ciclos no início da simulação.	grupo
5	Nível ativo e ordem dos bits	Todos os sinais ativos em **1**. Bit 7 = MSB, bit 0 = LSB. Barramentos escritos *[7:0]*.	grupo
6	Endereço da RAM	A RAM recebe **REM[3:0]** (A0 a A3). Os 4 bits altos do REM são ignorados.	M3
7	RDM tem duas fontes (RAM e AC)	RDM carrega quando **read OU cargaRDM**; a fonte é o AC quando cargaRDM = 1 e a RAM caso contrário. O mux fica dentro do módulo **rdm** (M2 usa o mux2x8 do M1). Sem tri-state.	M2 + M1
8	RI de 8 ou 4 bits	RI de 8 bits (mesmo reg8 dos outros); o opcode é **RI[7:4]**. Os 4 bits baixos são don't care.	M2
9	Opcodes inválidos	Tratar como NOP no decodificador, para a FSM nunca travar em t3.	M1

3. Documento de interfaces (contrato do grupo)

Esta tabela é a fonte única de verdade. **Ninguém muda nome, largura ou significado de um sinal sem avisar o grupo e atualizar o documento.** Os nomes abaixo devem aparecer exatamente assim nas portas dos módulos.

Sinal	Larg.	Origem → Destino	Descrição
`clk`	1	externo → todos	Registradores carregam na borda de subida.
`reset`	1	externo → todos	Síncrono, ativo em 1. Zera PC, AC, REM, RDM, RI, N, Z e o contador de tempo (volta a t0); recarrega a RAM com o programa.
`pc[7:0]`	8	PC → MUX REM	Endereço da próxima instrução.
`rdm[7:0]`	8	RDM → UAL(Y), MUX REM, PC, RI	Dado lido da RAM, operando ou endereço.
`ac[7:0]`	8	AC → UAL(X), MUX RDM	Acumulador.
`ual_out[7:0]`	8	UAL → AC	Resultado da operação.
`rem[7:0]`	8	REM → RAM	A RAM usa rem[3:0].
`mem_dout[7:0]`	8	RAM → RDM	Dado lido (vale 0 quando read = 0).
`sel`	1	UC → MUX REM	0 = PC, 1 = RDM.
`cargaREM, cargaRDM, cargaRI, cargaAC, cargaNZ, cargaPC`	1 cada	UC → registradores	Habilitam a carga do respectivo registrador.
`incrementaPC`	1	UC → PC	PC ← PC + 1.
`read, write`	1 cada	UC → RAM (read também → RDM)	read: RAM mostra o dado; write: grava rdm em RAM[rem].
`selUAL[2:0]`	3	UC → UAL	000 = X+Y, 001 = X AND Y, 010 = X OR Y, 011 = NOT X, 100 = Y.
`n, z`	1 cada	NZ → UC (e UAL → NZ na entrada)	N = ual_out[7]; Z = 1 se ual_out = 0. Só mudam com cargaNZ.
`opcode[3:0]`	4	RI[7:4] → DECOD	Código da instrução.
`i_nop, i_sta, i_lda, i_add, i_or, i_and, i_not, i_jmp, i_jn, i_jz, i_hlt`	1 cada	DECOD → UC	Uma linha ativa por instrução (one-hot).
`t0..t7`	1 cada	gerador_tempo → UC	Exatamente um ativo por ciclo.
`goto_t0, halt`	1 cada	UC → gerador_tempo	goto_t0 reinicia o ciclo; halt congela o contador.

Codificação das instruções (opcode em RI[7:4])

NOP	STA	LDA	ADD	OR	AND	NOT	JMP	JN	JZ	HLT
0000	0001	0010	0011	0100	0101	0110	1000	1001	1010	1111
1 byte	2 bytes	2 bytes	2 bytes	2 bytes	2 bytes	1 byte	2 bytes	2 bytes	2 bytes	1 byte
hex: 0x	1x	2x	3x	4x	5x	6x	8x	9x	Ax	Fx

Nos instruções de 2 bytes, o segundo byte é o endereço direto do operando. Os 4 bits baixos do primeiro byte valem 0. Exemplo: LDA 0E é gravado como 20 0E (hex).

Duração de cada instrução (ciclos de clock)

Instrução	Tempos usados	Ciclos
STA, LDA, ADD, OR, AND	t0 a t7	8
JMP, JN com N=1, JZ com Z=1 (desvio tomado)	t0 a t5	6
NOT, NOP, JN com N=0, JZ com Z=0 (não tomado), HLT	t0 a t3	4 (HLT fica parado em t3)

MEMBRO 1

Datapath combinacional

Biblioteca de portas · MUX 2:1 de 8 bits · UAL · decodificador de opcode

Você começa antes de todos: o grupo inteiro depende da sua **lib_portas.v**. Entregue-a no primeiro dia, mesmo que só com as portas básicas.

Entregáveis (nesta ordem)

1. **lib_portas.v**: not1, and2, and3, and4, or2, or3, or4, xor2, nor8 (para o Z) e mux2_1bit, todos montados com nand/nor/not. Peça aos colegas a lista de portas de que precisam.
2. **mux2x8**: portas *a[7:0]* (sel = 0), *b[7:0]* (sel = 1), *sel*, *y[7:0]*. A mesma peça serve para o MUX REM e para o MUX RDM.
3. **Somador**: meio-somador, somador completo e somador de 8 bits ripple-carry. O carry final é descartado (o Neander não tem flag de overflow ou carry).
4. **ual**: portas *x[7:0], y[7:0], selUAL[2:0] → out[7:0], n, z*.
5. **decod_opcode**: *opcode[3:0]* → 11 linhas i_nop … i_hlt, com opcodes inválidos tratados como NOP (i_nop = NOT(OR das outras 10)).

Como montar a UAL

- Calcule todos os resultados em paralelo: soma (X+Y), AND, OR, NOT X e a passagem de Y.
- Decodifique selUAL em 5 habilitações (000, 001, 010, 011, 100) e faça, para cada bit, *out[i] = OR de (habilitação_k AND resultado_k[i])*.
- **n = out[7]** e **z = NOR de out[7:0]** (use dois or4 e um nor).
- A operação Y (100) existe para o LDA atualizar N e Z passando o dado da memória pela UAL. Não esqueça dela.
- Combinações 101, 110 e 111: saída 0 (documente no comentário do módulo).

Testes mínimos (um tb por módulo)

- UAL: 00+00 = 00 (Z=1); 7F+01 = 80 (N=1); FF+01 = 00 (Z=1, N=0); NOT 00 = FF; NOT FF = 00 (Z=1); AND e OR com AA e 55; Y com 80 (N=1) e com 00 (Z=1).
- mux2x8: sel = 0 e sel = 1 com valores diferentes em a e b.
- decod_opcode: percorra os 16 opcodes; confira que só uma linha fica ativa em cada um.

Cuidados

- Combinacional puro: nada de latch ou flip-flop nos seus módulos.
- Combine com o M4 se o decodificador ficará separado ou dentro da UC (o contrato desta página assume separado).
- Depois que terminar, ajude o M5 a montar o modelo de referência da UAL e revise o código do M4.

Checklist

[ ] lib_portas.v entregue e testada
[ ] mux2x8 testado
[ ] somador 8 bits testado
[ ] UAL testada nos casos de borda
[ ] decod_opcode testado com os 16 opcodes
[ ] Interfaces batem com a seção 3

MEMBRO 2

Registradores e contador PC

Latch · flip-flop · AC, REM, RDM, RI · NZ · PC com carga e incremento

Todo o armazenamento do processador (menos a RAM) sai daqui. O enunciado diz que AC, REM, RDM e RI são iguais: faça **um só módulo reg8** e reutilize.

Entregáveis (nesta ordem)

1. **dlatch** (latch D com NAND) e **dff** (mestre-escravo, borda de subida).
2. **reg1**: um bit com carga e reset síncrono. Entrada efetiva: *d_ef = reset ? 0 : (load ? d : q)*, montada com portas.
3. **reg8**: portas *clk, reset, load, d[7:0] → q[7:0]*. Usado como AC, REM e RI.
4. **rdm**: um reg8 com *load = read OR cargaRDM* e entrada *mux2x8(sel = cargaRDM, a = mem_dout, b = ac)*. Portas: *clk, reset, read, cargaRDM, mem_dout, ac → rdm*.
5. **regnz**: 2 bits. Portas *clk, reset, cargaNZ, n_in, z_in → n, z*. Reset zera os dois.
6. **pc8**: contador de 8 bits. Portas *clk, reset, cargaPC, incrementaPC, d[7:0]* (vem do RDM) *→ q[7:0]*. Prioridade: reset > cargaPC > incrementaPC. Incremento: o bit *i* inverte quando todos os bits abaixo dele valem 1 (cadeia de meios-somadores).

Testes mínimos

- reg8: com load = 0 o valor não muda; com load = 1 carrega só na borda de subida; reset zera mesmo com load = 1.
- rdm: carrega da RAM quando read = 1; carrega do AC quando cargaRDM = 1; mantém quando os dois são 0.
- pc8: incrementa 00 → 01 → … ; vira de FF para 00; carrega um valor e depois incrementa; reset limpa.
- regnz: carrega os dois flags juntos; não muda sem cargaNZ.

Cuidados

- **Valores X no início da simulação.** Os latches de NAND começam indefinidos. Com o reset passando pelo mux de entrada, o primeiro ciclo de clock resolve o estado, por isso mantenha reset = 1 por pelo menos 2 ciclos na abertura de toda testbench.
- Todos os FFs usam o mesmo clk. Como a saída só muda na borda, encadear AC → RDM → RI não gera corrida (se o mestre-escravo estiver certo). Teste isso explicitamente: um reg8 ligado na saída de outro.
- Entregue o **reg1/dff** cedo ao M3 e ao M4 (contador de tempo): eles reutilizam sua peça.

Checklist

[ ] dlatch e dff testados
[ ] reg8 com reset e load testado
[ ] rdm com as duas fontes testado
[ ] regnz testado
[ ] pc8 com prioridade reset > carga > incremento
[ ] dff/reg1 entregue para M3 e M4

MEMBRO 3

Memória RAM 16 × 8

Banco de registradores · decodificadores · leitura/escrita · RESET carrega o programa

A RAM é um grande banco de 16 registradores de 8 bits, com decodificadores de endereço e seletores de saída, como na figura do enunciado (decodificadores X e Y de 2 → 4 formando uma matriz 4 × 4, ou um decodificador único 4 → 16: ambos servem).

Interface

```
ram16x8( clk, reset, addr[3:0], din[7:0], read, write, dout[7:0], dbg_mem[127:0] )
addr = rem[3:0] din = rdm dout → mem_dout (vale 0 quando read = 0)
dbg_mem: as 16 posições concatenadas, só para a testbench conferir o conteúdo
```

Entregáveis (nesta ordem)

1. **Decodificador** 4 → 16 (ou X e Y de 2 → 4) gerando *sel_0 … sel_15*.
2. **Célula de bit** com flip-flop, em duas versões: INIT = 0 e INIT = 1. Entrada efetiva: *d_ef = reset ? INIT : (we ? din : q)*. Para INIT = 0 é um AND com NOT reset; para INIT = 1 é um OR com reset.
3. **Byte** = 8 células, com *we = write AND sel_i* e a constante INIT de cada bit vinda da tabela do programa.
4. **Leitura**: para cada bit, *dout[b] = read AND (OR de sel_i AND q_i[b], i = 0..15)* (árvore AND-OR).
5. **mem_init**: um único ponto com os 16 valores iniciais (parâmetros). O M5 preenche; você só ajusta a ligação.

Testes mínimos

- Escreva um valor distinto em cada um dos 16 endereços e leia todos de volta.
- Escrever no endereço A não altera nenhum outro endereço.
- Com read = 0, dout = 00. Com write = 1 e read = 0, grava sem mostrar saída.
- Depois do reset, a RAM é idêntica à tabela do programa (use dbg_mem). Reset tem prioridade sobre write.

Cuidados

- A escrita é síncrona: o processador mantém write = 1 durante todo t7 e a gravação acontece na borda de subida que encerra esse ciclo.
- Leitura combinacional: quando REM muda em t0, em t1 (read = 1) o dado já está disponível em dout; o RDM captura na borda seguinte.
- Memória com 16 posições, mas o endereço do programa vem em 8 bits: só os 4 bits baixos importam.
- Se o grupo preferir o simulador de portas em vez de Verilog, a mesma arquitetura vale; só troque as constantes INIT por pinos de preset.

Checklist

[ ] Decodificador testado
[ ] Célula INIT=0 e INIT=1 testadas
[ ] 16 bytes ligados e leitura AND-OR testada
[ ] Reset carrega o programa (conferido via dbg_mem)
[ ] read=0 → dout=0
[ ] Interface bate com a seção 3

MEMBRO 4

Unidade de Controle

Gerador de tempos t0–t7 · equações dos sinais · FSM do processador

A UC é uma máquina de estados: um contador de tempo (t0 a t7) mais lógica combinacional que, olhando a instrução decodificada e os flags, liga os sinais de controle de cada tempo. Base: Tabelas 10.2, 10.3 e 10.4 e as equações da página 10-8 do PDF de organização.

Entregáveis (nesta ordem)

1. **gerador_tempo**: contador de 3 bits (use o dff do M2) + decodificador 3 → 8. Portas: *clk, reset, goto_t0, halt → t0 … t7*. Regra de próximo estado: se reset ou goto_t0 → 000; senão, se halt → mantém; senão soma 1.
2. **unidade_controle**: entradas *t0…t7, i_*, n, z*; saídas *sel, cargaREM, cargaRDM, cargaRI, cargaAC, cargaNZ, cargaPC, incrementaPC, read, write, selUAL[2:0], goto_t0, halt*.
3. Testbench que força cada instrução e cada tempo e compara os sinais com as Tabelas 10.3 e 10.4.

Equações (as do PDF, com os grupos nomeados)

Defina dois grupos para ficar legível: **MEMOP** = STA + LDA + ADD + OR + AND e **TOMADO** = JMP + JN·N + JZ·Z (desvio realizado). **NTOMADO** = JN·N' + JZ·Z'.

```
cargaREM = t0 + t3·(MEMOP + TOMADO) + t5·MEMOP
incrementaPC = t1 + t4·MEMOP + t3·NTOMADO
cargaRI = t2
sel = t5·MEMOP
cargaRDM = t6·STA
read = t1 + t4·(MEMOP + TOMADO) + t6·(LDA+ADD+OR+AND)
write = t7·STA
cargaAC = cargaNZ = t7·(LDA+ADD+OR+AND) + t3·NOT
cargaPC = t5·TOMADO
goto_t0 = t7·MEMOP + t3·(NOP + NOT + NTOMADO) + t5·TOMADO
halt = t3·HLT
```

Sinais da UAL. Gere primeiro UAL(Y) = t7·LDA, UAL(ADD) = t7·ADD, UAL(OR) = t7·OR, UAL(AND) = t7·AND, UAL(NOT) = t3·NOT. Depois codifique: `selUAL[2] = UAL(Y)`, `selUAL[1] = UAL(OR) + UAL(NOT)`, `selUAL[0] = UAL(AND) + UAL(NOT)` (ADD = 000).

Cuidados

- **A equação do PDF original tem um parêntese sem fechar em cargaREM.** Use a versão acima.
- Em **t0, t1 e t2 nenhuma equação pode depender de i_***: o RI só é carregado no fim de t2, então o decodificador mostra a instrução anterior até t3. A tabela acima já respeita isso.
- HLT: em t3 ativa-se halt e o contador congela em t3. Nesse tempo nenhuma outra equação liga sinais (confira), então o processador fica parado de forma segura.
- Reset deixa o RI = 00, ou seja, NOP, e o contador em t0.
- Teste JN e JZ com flag = 0 e com flag = 1: tomado (6 ciclos) e não tomado (4 ciclos).

Checklist

- [ ] gerador_tempo: 1 ativo por ciclo, goto_t0 volta a t0, halt congela, reset volta a t0
- [ ] Cada instrução confere com as Tabelas 10.3 e 10.4
- [ ] JN e JZ testados tomados e não tomados
- [ ] selUAL codificado como na seção 3
- [ ] Equações revisadas por outro membro

MEMBRO 5

Programa, integração e simulação

Programa de 10 instruções · modelo de referência · top-level · testbench · relatório

Você é o dono do resultado final: o que o professor vê é o seu top-level rodando o seu programa. Comece cedo (modelo de referência e programa) e integre à medida que os módulos chegam.

Entregáveis

1. **Pergunta ao professor** (ver seção 5) e registro da resposta.
2. **Programa** de 10 instruções, diferente dos outros grupos, e a tabela hexadecimal de 16 posições (**mem_init**) para o M3.
3. **Modelo de referência** em Python (ou outra linguagem): executa o programa instrução por instrução e imprime PC, AC, N, Z e memória esperados.
4. **neander_top.v**: instancia PC, MUX REM, REM, RAM, RDM, AC, UAL, NZ, RI, DECOD e UC, ligados exatamente como a seção 3. Expõe *pc, ac, n, z, t, halt* para depuração.
5. **tb_neander.v**: gera clock, aplica reset por 2 ciclos, roda até halt (com timeout), grava VCD (*$dumpvars*), imprime uma linha por instrução em t0 e despeja a memória no final.
6. **Relatório** com diagrama, formas de onda, tabela esperado × obtido e a explicação de cada módulo.

Orçamento de bytes do programa

Use a conta da seção 2: **bytes de código = 10 + x** (x = número de instruções de 2 bytes), **bytes de dados = 6 − x**. Para cobrir bem o processador, o programa deveria ter: LDA e STA, uma operação aritmética (ADD) e uma lógica (AND ou OR), NOT, um desvio condicional **tomado**, outro **não tomado** e HLT.

Exemplo de formato (para vocês criarem um programa próprio)

Mostra como escrever a tabela e o resultado esperado. **Não entreguem este mesmo programa**: o enunciado pede um programa diferente por grupo. Aqui x = 4, então são 14 bytes de código e 2 bytes de dados (E e F).

End.	Hex	Instrução	Efeito
0, 1	20 0E	LDA 0E	AC ← MEM[0E] = 05 (N=0, Z=0)
2	60	NOT	AC = FA (N=1)
3, 4	30 0F	ADD 0F	AC = FA + 01 = FB (N=1): é −5
5, 6	90 08	JN 08	N=1, desvio tomado para o endereço 08
7	F0	HLT	pulado pelo desvio
8, 9	10 0F	STA 0F	MEM[0F] ← FB
A	60	NOT	AC = 04 (N=0)
B	60	NOT	AC = FB (N=1)
C	00	NOP	—
D	F0	HLT	para; PC = 0E
E	05	dado A	valor inicial 05
F	01	dado B	valor inicial 01 (depois vira FB)

Esse exemplo executa 9 instruções e gasta **50 ciclos** (8+4+8+6+8+4+4+4+4). Estado final esperado: AC = FB, N = 1, Z = 0, MEM[0F] = FB. Use essa conta para calibrar o seu timeout e para conferir o modelo de referência.

Ordem de integração (testes)

1. **Datapath + RAM sem UC**: force à mão os sinais de controle e execute um LDA e um STA completos.
2. **UC ligada**: mini-programas de teste, um por instrução (NOP, NOT, ADD, AND, OR, LDA/STA), mais JMP, JN e JZ tomados e não tomados e o HLT.
3. **Programa final**: compare cada linha da simulação com o modelo de referência. Guarde a forma de onda para o relatório.

Checklist

- [ ] Resposta do professor sobre instruções × bytes e sobre Verilog × simulador
- [ ] Programa diferente dos outros grupos e tabela mem_init entregue ao M3
- [ ] Modelo de referência gera o trace esperado
- [ ] neander_top ligado conforme a seção 3
- [ ] Testbench roda até halt e despeja a memória
- [ ] Trace obtido = trace esperado
- [ ] Relatório com formas de onda

4. Cronograma e marcos

Sem datas fixas: ajustem as fases ao prazo de vocês. Cada fase só termina quando o critério de pronto é verdadeiro.

Fase	O que acontece	Quem	Critério de pronto
0 Dia 1	Reunião, decisões da seção 2, perguntas ao professor, repositório criado. M1 entrega lib_portas.v.	todos; M1; M5	Seção 3 aceita por todos; lib_portas.v no repositório.
1 Módulos	Cada um constrói e testa seus módulos isoladamente.	M1, M2, M3, M4	Cada módulo com testbench passando.
2 Datapath	Datapath + RAM ligados no top-level, sem UC. LDA e STA manuais.	M5 com M1, M2, M3	Valor sai da RAM, passa pelo AC e volta à RAM.
3 Controle	UC ligada. Cada instrução testada, inclusive JN/JZ tomados e não tomados e HLT.	M4 com M5	Cada instrução igual às Tabelas 10.3/10.4.
4 Programa	Programa final rodando e conferido com o modelo de referência.	M5 com todos	Trace obtido = trace esperado; AC, N, Z e memória finais corretos.
5 Entrega	Relatório, formas de onda, revisão cruzada, arquivo final.	todos	Checklist da seção 5 completo.

Convenções de projeto

- **Um módulo por arquivo**, com o nome do módulo: `lib_portas.v`, `mux2x8.v`, `ual.v`, `decod_opcode.v`, `reg8.v`, `rdm.v`, `regnz.v`, `pc8.v`, `ram16x8.v`, `gerador_tempo.v`, `unidade_controle.v`, `neander_top.v`.
- Testbench de cada módulo no repositório (`tb_<modulo>.v`), com Git ou pasta versionada. Ninguém altera módulo de outro: avise o dono.
- **Revisão cruzada** antes de integrar: M1 revisa M4, M2 revisa M3, M3 revisa M2, M4 revisa M1, M5 revisa a ligação de todos.

5. Entrega final e perguntas ao professor

Checklist geral

- [ ] Todos os módulos do processador usam só nand, nor, not, latches e flip-flops (conforme resposta do professor)
- [ ] Passos 1 a 4 do enunciado atendidos: combinacionais, sequenciais, unidade de controle e programa
- [ ] RESET inicializa PC, AC, N, Z, tempo, RAM (com o programa) e deixa o RI em NOP
- [ ] Programa de 10 instruções em 0000..1010 e dados nos endereços restantes
- [ ] Simulação mostra o programa rodando do reset até o HLT, com a conferência de cada instrução
- [ ] Relatório com diagrama de blocos, formas de onda e esperado × obtido

Perguntas para mandar ao professor

1. As "10 instruções" são 10 instruções ou 10 bytes de código? O enunciado diz endereços 0000 a 1010 (11 posições) e 6 endereços para dados, o que não fecha com 16 posições.
2. Precisamos usar um simulador lógico de portas ou Verilog estrutural basta? Em Verilog, as primitivas and/or/xor são permitidas, ou apenas nand, nor e not?
3. Podemos usar testbench comportamental e modelo de referência? Qual o formato de entrega e o prazo?